

Tries

James Anderson (Modified by Cemil Kirbas)

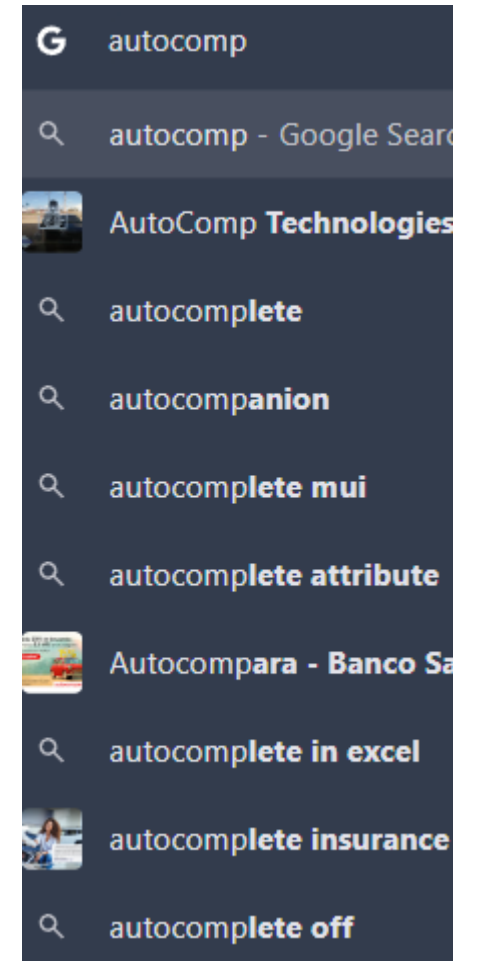
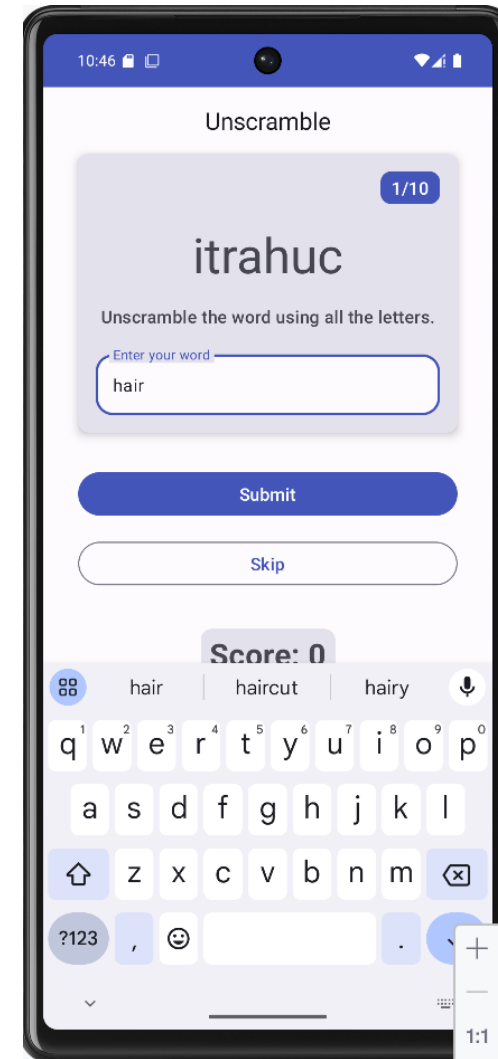
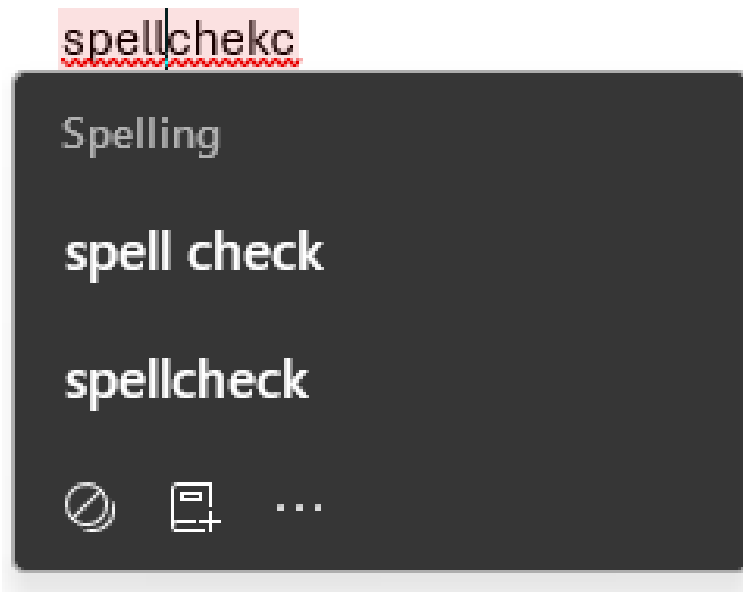
Data Structures and Algorithms

CS 3100/5100

Wright State University

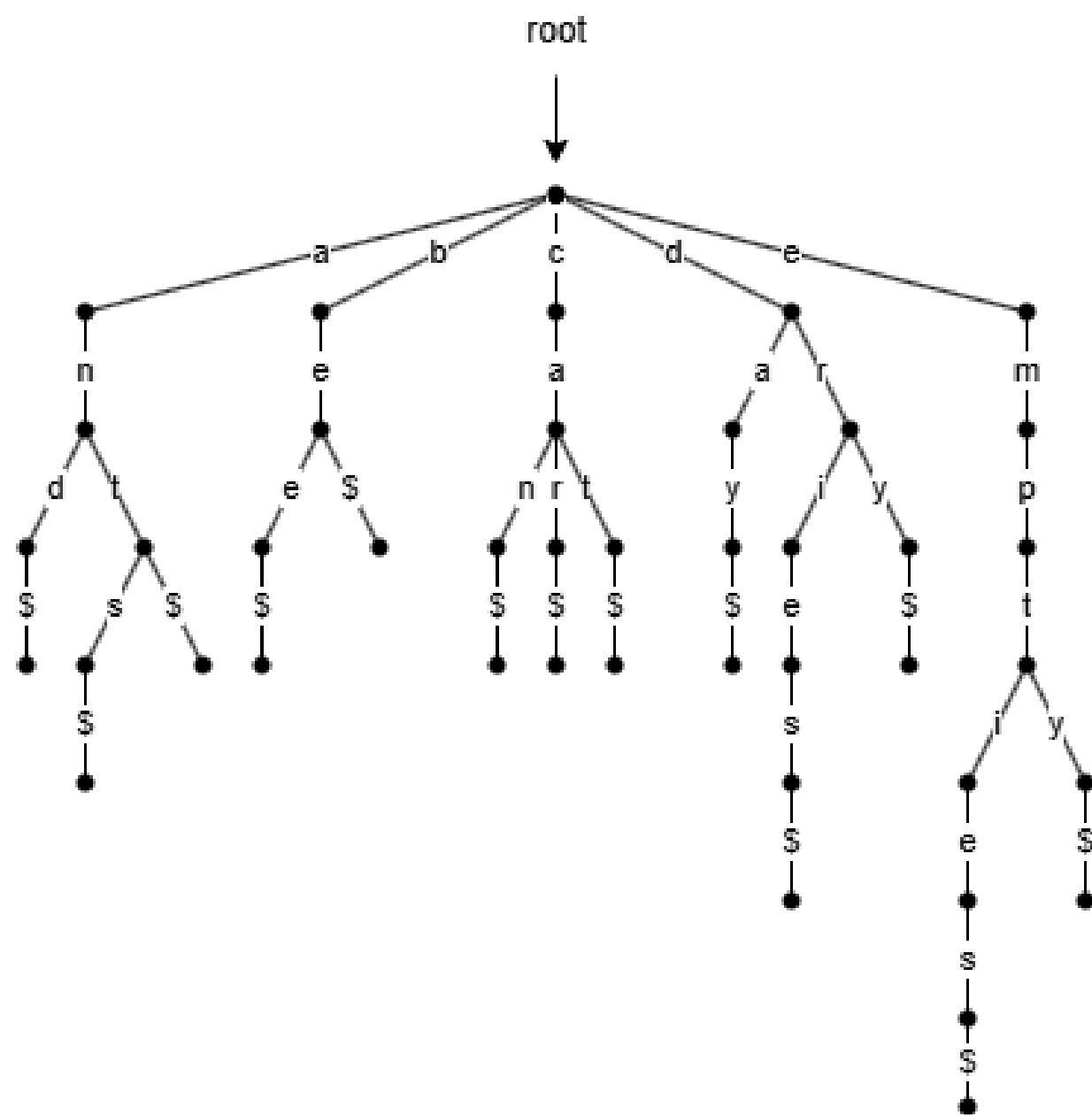
Motivation

- Tries are data structure for storing strings
- The word trie comes from **retrieval**
- Spellcheck
- Autocomplete

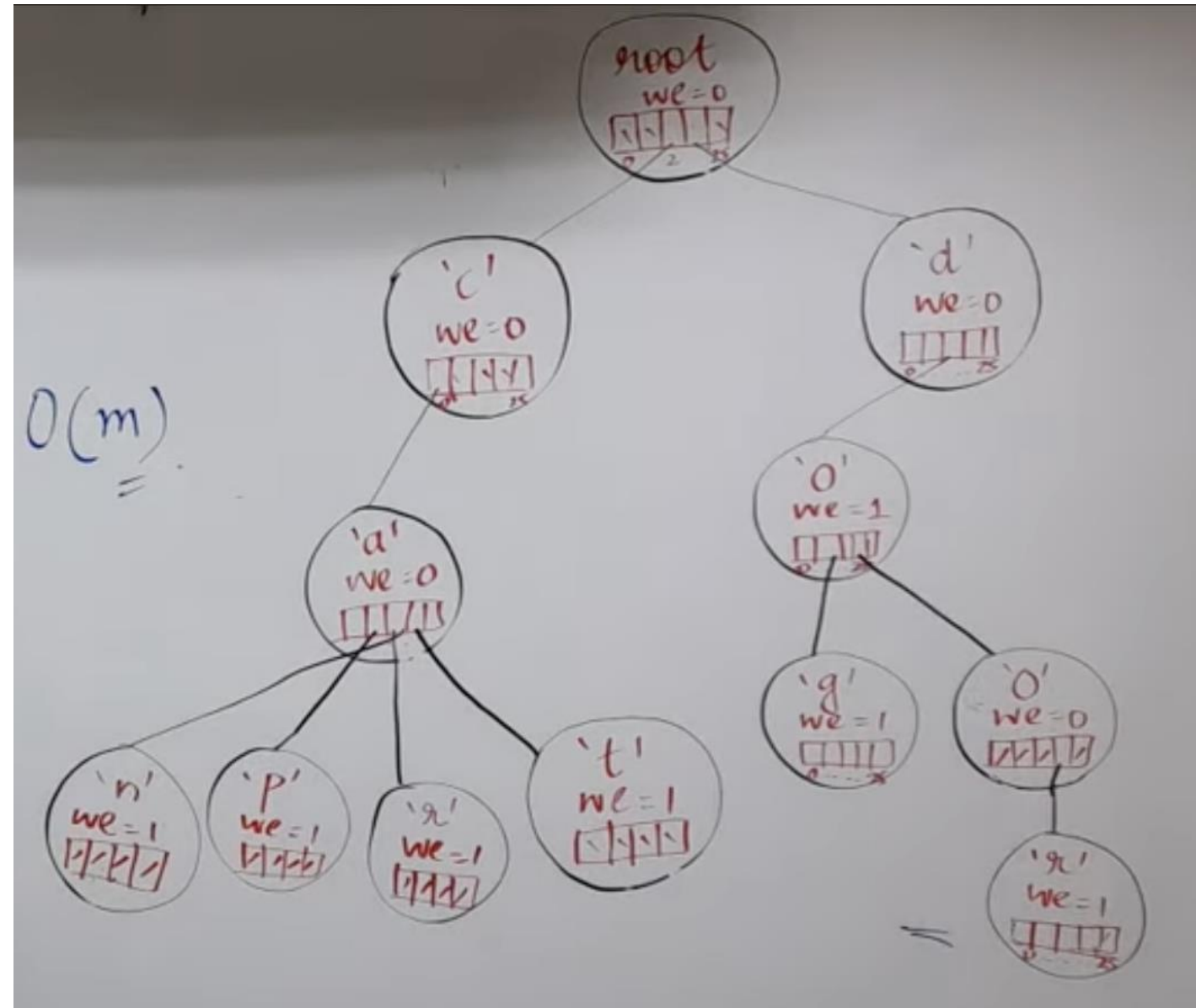
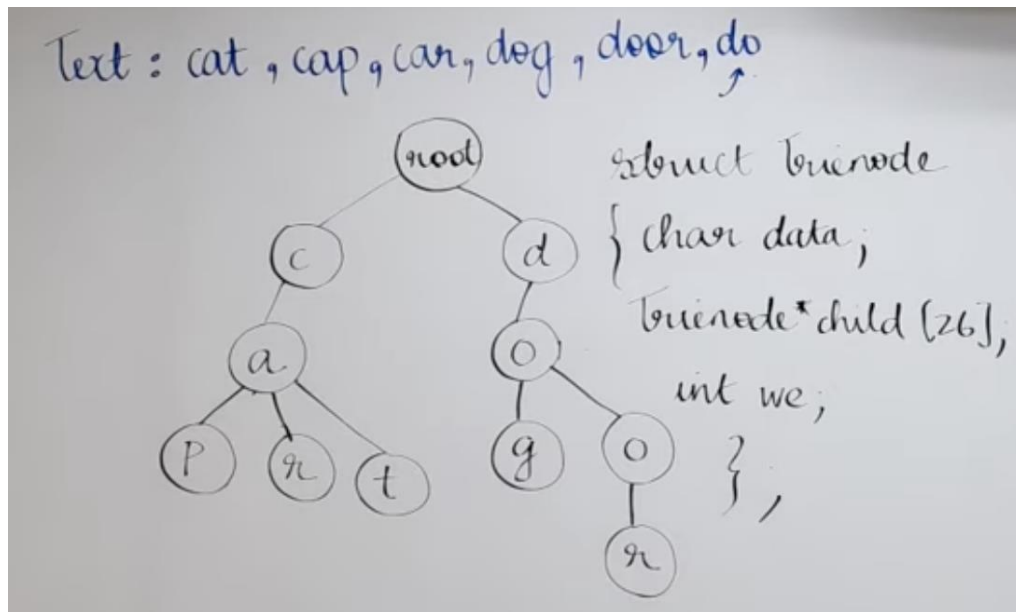


Alphabet Trie

- Branching factor can be greater than 2
- Nodes with 26 children – one for each letter a to z
- End of word depicted by a terminal node
- Pointers create a word prefix, subtrees contain all words begin with that prefix
- Also called prefix tree



Tries



Trie Insert

- Loop through string, for each character C
 - If no child for C, add a new child
 - Otherwise, assign current to the child pointer associated with C

Trie Search

- Loop through all characters C in key string
- If current node has child for C, set current to child pointer
- If current node is end of word, word found, return true
- If current node is ever null, word not in trie, return false

Trie Remove

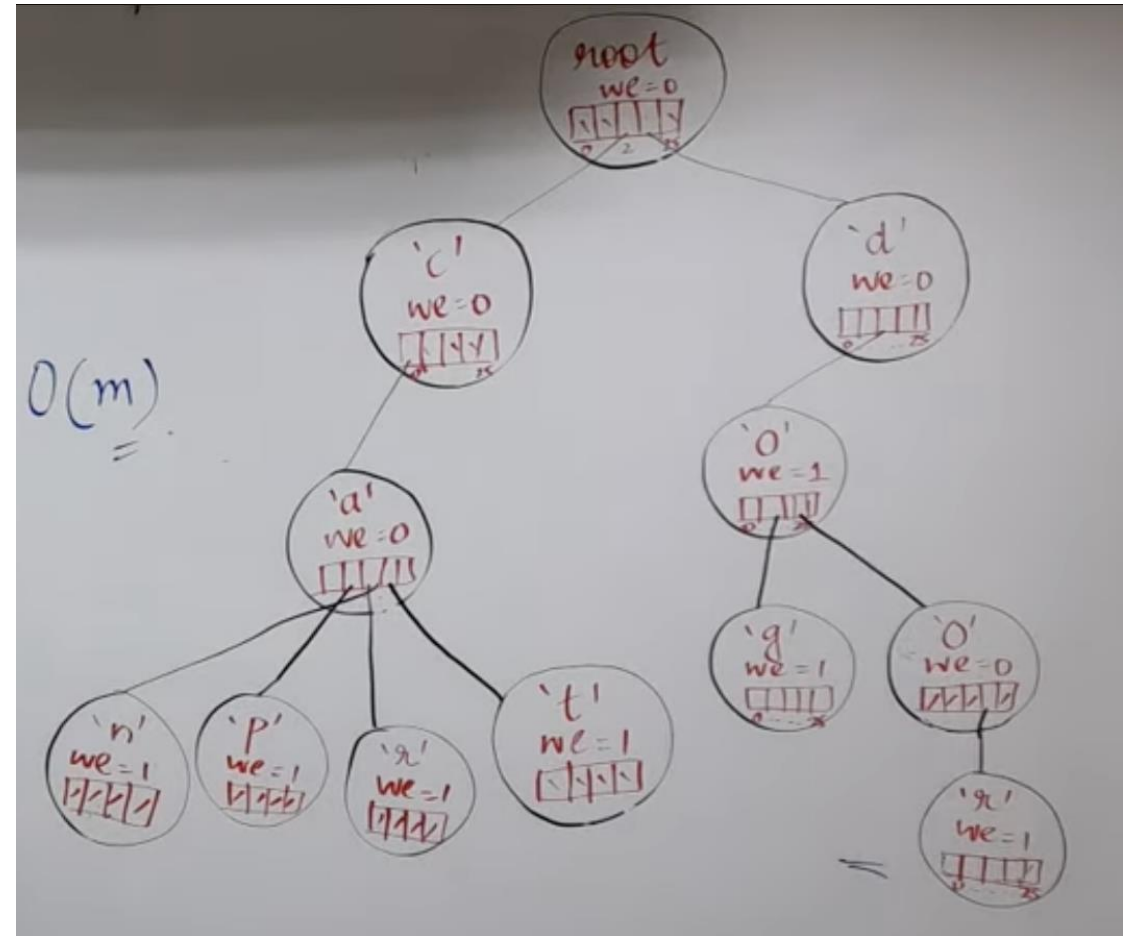
- Remove end of word node for given word
- Traverse back up and remove nodes until finding one with more than one child – that node has more words below it

Note on Notation

- The book depicts tries slightly different than I do
- My tries: character on the edge, signifying the pointer is to a node that has all words starting (or continuing) from that letter
- Book tries: character in the node – this is the node pointed to that has all words starting (or continuing) from that letter
- Unlike other trees (BST, AVLTree), an empty trie has a root node, the root node contains all null pointers

Tries

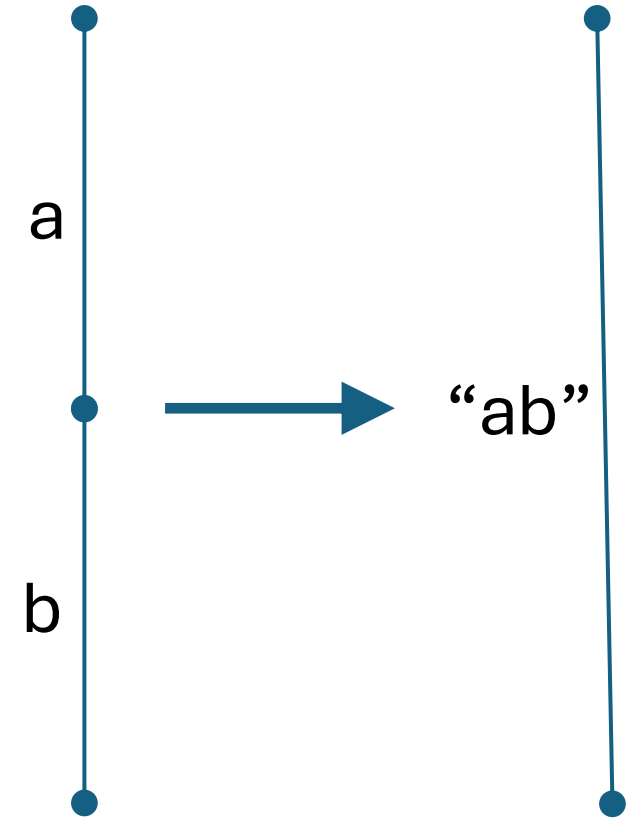
- Time complexity of insert, search and remove is $O(m)$, where m is the length of the word



- Space complexity of tries is high
- Space complexity is $O(\text{\#of text}) - O(n)$ where n is the sum of the number of characters in each word.

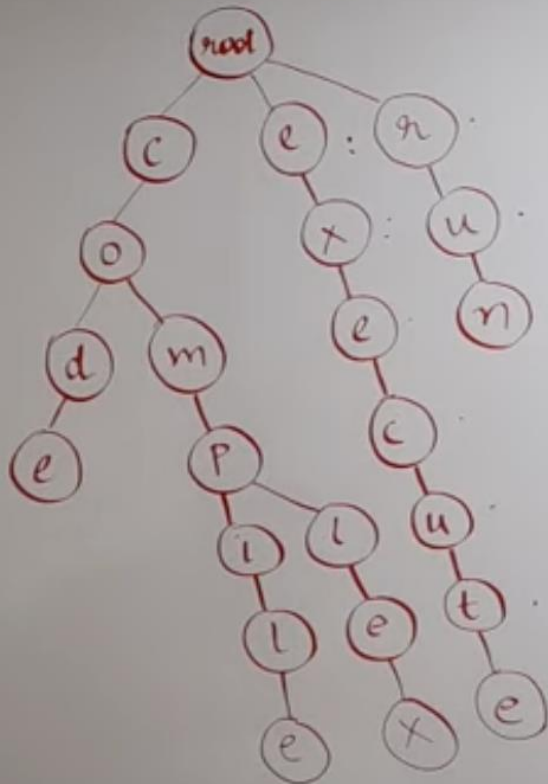
Compressed Trie

- Many English words share the same prefix with multiple endings
- E.g. program: programed, programmer, programmers, programmable, programmables, programmed, programming, programs
- If there is only one letter that will appear after another in any given word, we can compress the nodes into one
- A trie that each node, except leaf nodes, has at least two children.

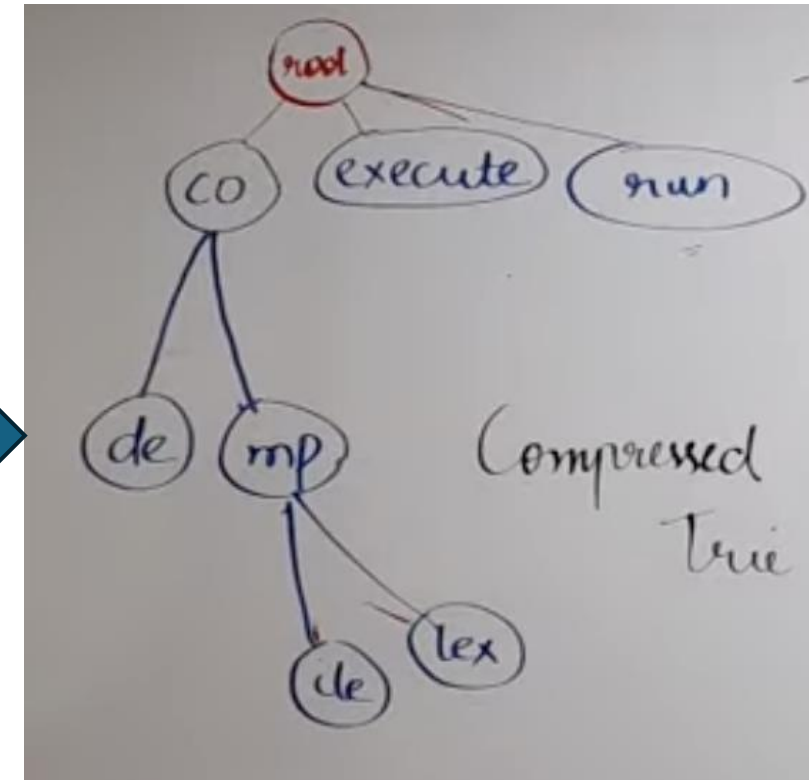


Compressed Trie

- Starting from the leaf node, collapse the nodes that have only one child with its child node.
- None of the words should be a prefix of another word.

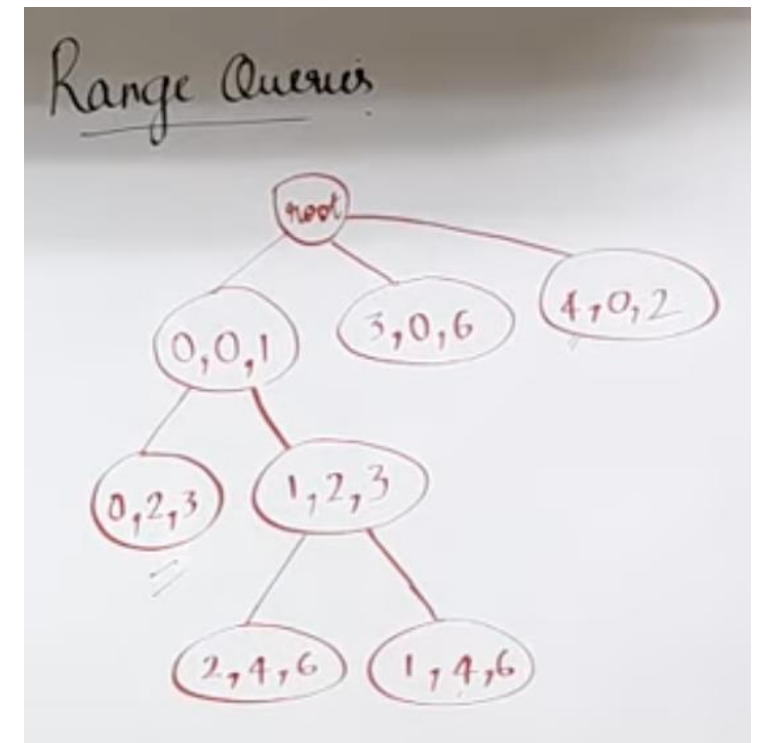
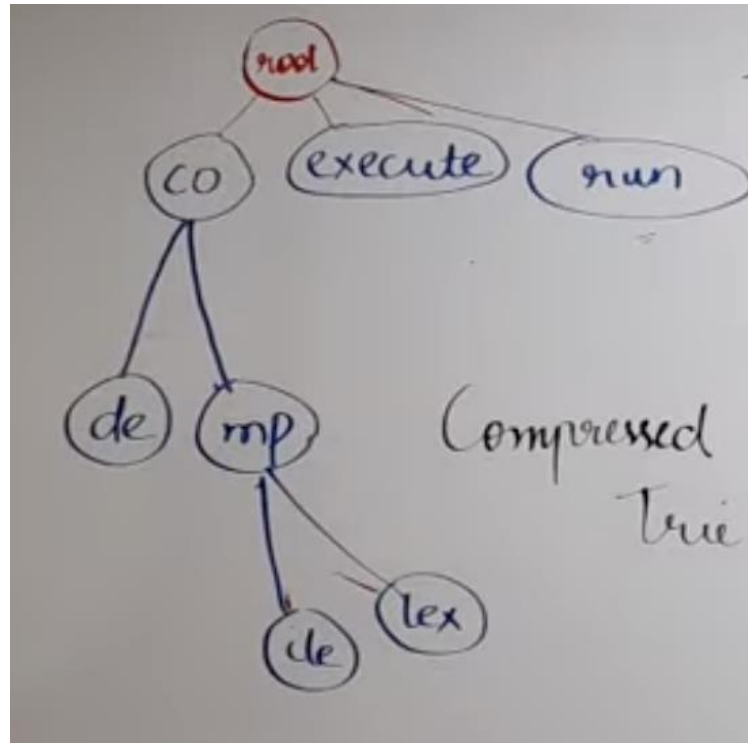


Text: c o d e
c o m p l e x
c o m p i l e
e x e c u t e
r u n



Compressed Trie

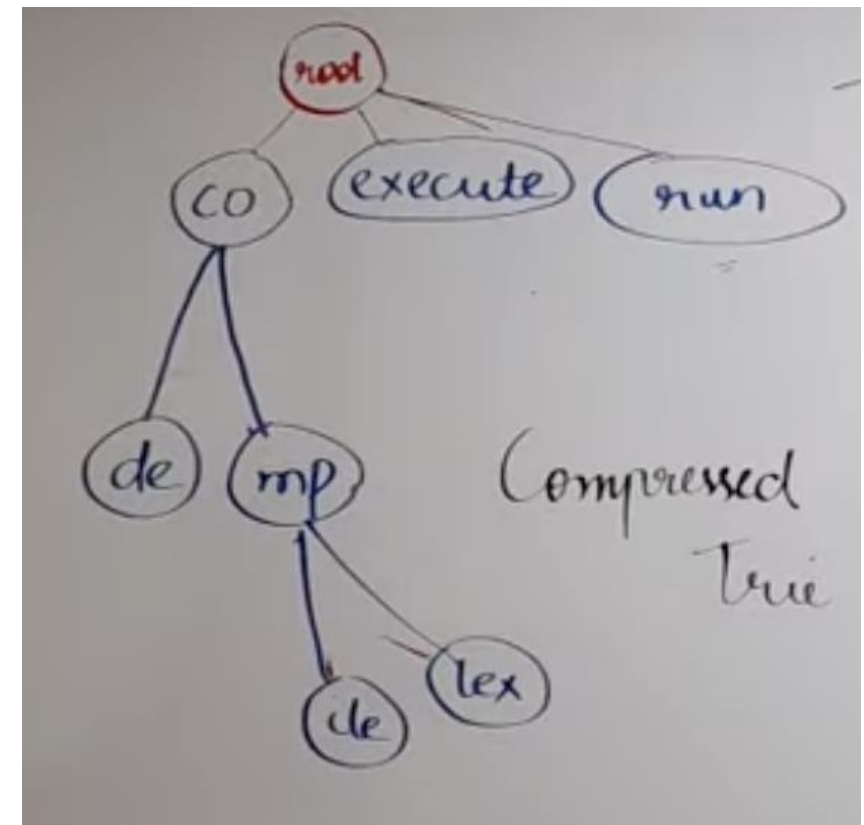
Text: code
complex
compile
execute
run



- None of the words should be a prefix of another word.
- A trie in which every node has at least two children, the number of internal nodes is at most $L-1$, where L is the number of leaf nodes.
- #of leaf nodes = # of words
- Total # of nodes = $2 \times \text{\#of words}$
- Space complexity is $O(\text{\#of words})$

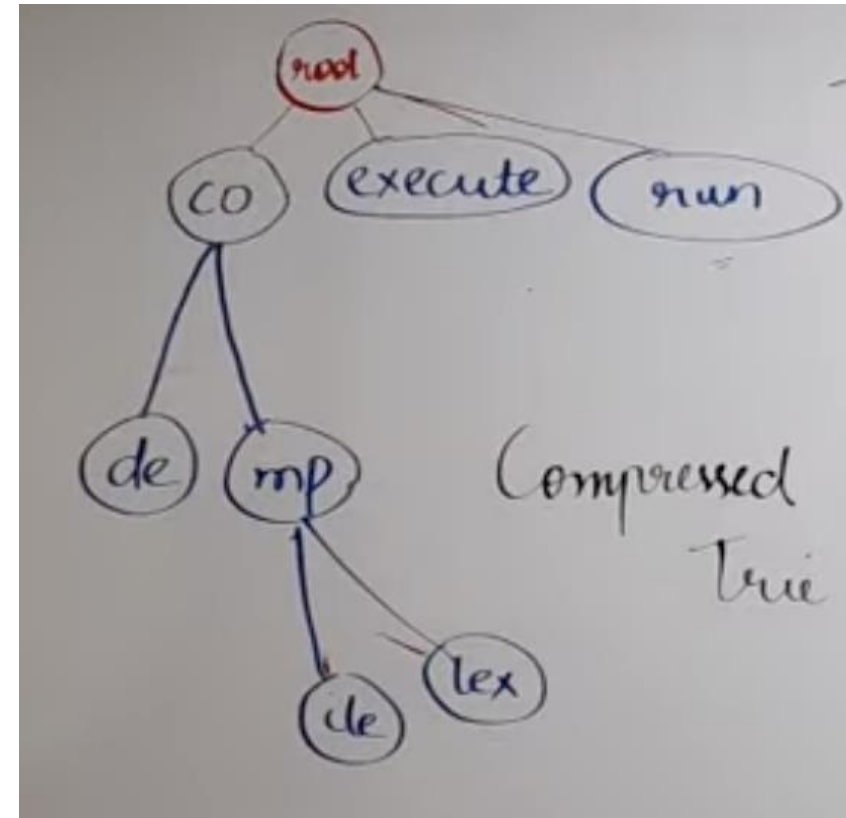
Compressed Trie Insert

- Options:
 - Insert like a regular trie, compress
 - Compress on each insert
- Result in same trie
- Compress on insert:
 - Find how much of word matches the substring in trie node
 - If needed, create new branch
 - Insert rest of word into node's subtree



Compressed Trie Search

- Get node associated with first letter of word
- Determine if prefix matches beginning of word
- Search that node with the remaining letters



Compressed Trie Remove

- Find word in trie
- Remove end of word
- If there is nothing branching off current node, delete and add substring into parent

